

CONFIGMAP YAML FILE

A ConfigMap is a Kubernetes object used to store non-confidential configuration data such as application settings, environment variables, URLs, or feature flags. This data is stored in plain text and can be consumed by Pods.

- 1) Clone the Repository:

git clone https://github.com/Msocial123/fss-Retail-App_kubernetes.git

```
root@ip-172-31-17-28:/home/ubuntu# git clone https://github.com/Msocial123/fss-Retail-App_kubernetes.git
Cloning into 'fss-Retail-App_kubernetes' ...
remote: Enumerating objects: 2432, done.
remote: Counting objects: 100% (153/153), done.
remote: Compressing objects: 100% (109/109), done.
remote: Total 2432 (delta 84), reused 54 (delta 40), pack-reused 2279 (from 2)
Receiving objects: 100% (2432/2432), 9.40 MiB | 7.68 MiB/s, done.
Resolving deltas: 100% (552/552), done.
root@ip-172-31-17-28:/home/ubuntu#
```

- 2) Change the current working directory:

cd fss-Retail-App_Kubernetes

cd k8s-manifests

```
root@ip-172-31-17-28:/home/ubuntu# ls
fss-Retail-App_kubernetes
root@ip-172-31-17-28:/home/ubuntu# cd fss-Retail-App_kubernetes/
root@ip-172-31-17-28:/home/ubuntu/fss-Retail-App_kubernetes# ls
Dockerfile  Notes  Public  README.md  docker-compose.yaml  k8s-manifests  node_modules  package-lock.json  package.json  server.js  test-volumes.yaml
root@ip-172-31-17-28:/home/ubuntu/fss-Retail-App_kubernetes# cd k8s-manifests/
root@ip-172-31-17-28:/home/ubuntu/fss-Retail-App_kubernetes/k8s-manifests# ls
mongodb-deployment.yaml  mongodb-service.yaml  usernode-js-service.yaml  userprofile-deployment.yaml
root@ip-172-31-17-28:/home/ubuntu/fss-Retail-App_kubernetes/k8s-manifests#
```

- 3) Move the existing yaml files to new retail-app-deployment.yaml and retail-app-service.yaml files

```
root@ip-172-31-17-28:/home/ubuntu/fss-Retail-App_kubernetes/k8s-manifests# mv userprofile-deployment.yaml retail-app-deployment.yaml
root@ip-172-31-17-28:/home/ubuntu/fss-Retail-App_kubernetes/k8s-manifests# mv usernode-js-service.yaml retail-app-service.yaml
root@ip-172-31-17-28:/home/ubuntu/fss-Retail-App_kubernetes/k8s-manifests# ls
aws  awscli2.zip  kubectl  kubectl.sha256  mongodb-deployment.yaml  mongodb-service.yaml  retail-app-deployment.yaml  retail-app-service.yaml
root@ip-172-31-17-28:/home/ubuntu/fss-Retail-App_kubernetes/k8s-manifests#
```

4) Create a new namespace:

Kubectl create ns rithika-ns

```
root@ip-172-31-17-28:/home/ubuntu/fss-Retail-App_kubernetes/k8s-manifests# kubectl create ns rithika-ns
namespace/rithika-ns created
```

5) Create ConfigMap yaml file:

Vi configmap.yaml

Then apply this file by ,

Kubectl apply -f configmap.yaml

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: retail-app-config
  namespace: rithika-ns
data:
  MONGODB_URI: "mongodb://mongodb:27017/myDatabase"
  SESSION_SECRET: "1234"
  PORT: "3130"
  MONGO_INITDB_DATABASE: "myDatabase"
```

6) Edit yaml files:

Remove the data from the existing file and add new codes.

Vi retail-app-service.yaml

***retail-app-service.yaml:**

```
apiVersion: v1
kind: Service
metadata:
  name: retail-app-service
  namespace: rithika-ns
spec:
  type: LoadBalancer # Use NodePort if LoadBalancer is not supported
  ports:
    - port: 3130
      targetPort: 3130
      protocol: TCP
  selector:
    app: retail
```

Vi retail-app-deployment.yaml

*retail-app-deployment.yaml:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: retail-app-deployment
  namespace: rithikaa-ns
spec:
  replicas: 4
  selector:
    matchLabels:
      app: retail
  template:
    metadata:
      labels:
        app: retail
    spec:
      containers:
        - name: retail-app-container
          image: saiteja2502/userprofileretail:latest
          ports:
            - containerPort: 3130 # Change to match your app's port
          env:
            - name: MONGODB_URI
              valueFrom:
                configMapKeyRef:
                  name: retail-app-config
                  key: MONGODB_URI
            - name: SESSION_SECRET
              valueFrom:
                configMapKeyRef:
                  name: retail-app-config
                  key: SESSION_SECRET
            - name: PORT
              valueFrom:
                configMapKeyRef:
                  name: retail-app-config
                  key: PORT
            - name: MONGO_INITDB_DATABASE
              valueFrom:
                configMapKeyRef:
                  name: retail-app-config
```

```

        key: MONGO_INITDB_DATABASE
- name: EMAIL_USER
  valueFrom:
    secretKeyRef:
      name: retail-app-secret
      key: EMAIL_USER
- name: EMAIL_PASS
  valueFrom:
    secretKeyRef:
      name: retail-app-secret
      key: EMAIL_PASS

```

Vi mongodb-deployment.yaml

***mongodb-deployment.yaml:**

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: retail-mongodb
  namespace: rithika-ns
spec:
  replicas: 1
  selector:
    matchLabels:
      app: mongodb
  template:
    metadata:
      labels:
        app: mongodb
    spec:
      containers:
        - name: mongodb
          image: mongo:latest
          ports:
            - containerPort: 27017
          env:
            - name: MONGO_INITDB_DATABASE
              value: "myDatabase"

```

Vi mongodb-service.yaml

*mongodb-service.yaml:

```
apiVersion: v1
kind: Service
metadata:
  name: mongodb
  namespace: rithika-ns
spec:
  selector:
    app: mongodb
  ports:
    - port: 27017
      targetPort: 27017
      protocol: TCP
```

7) Apply all the files:

```
root@ip-172-31-17-28:/home/ubuntu/fss-Retail-App_kubernetes/k8s-manifests# kubectl apply -f configmap.yaml
configmap/retail-app-config created
root@ip-172-31-17-28:/home/ubuntu/fss-Retail-App_kubernetes/k8s-manifests# kubectl apply -f mongodb-deployment.yaml
deployment.apps/retail-mongodb created
root@ip-172-31-17-28:/home/ubuntu/fss-Retail-App_kubernetes/k8s-manifests# kubectl apply -f mongodb-service.yaml
service/mongodb created
root@ip-172-31-17-28:/home/ubuntu/fss-Retail-App_kubernetes/k8s-manifests# kubectl apply -f retail-app-deployment.yaml
deployment.apps/retail-app-deployment created
root@ip-172-31-17-28:/home/ubuntu/fss-Retail-App_kubernetes/k8s-manifests# kubectl apply -f retail-app-service.yaml
service/retail-app-service created
```

8) Check the status of the application:

```
root@ip-172-31-17-28:/home/ubuntu/fss-Retail-App_kubernetes/k8s-manifests# kubectl get all -n rithika-ns
```

NAME	READY	STATUS	RESTARTS	Age
pod/retail-app-deployment-68b685c7f6-5l6ps	1/1	Running	0	61s
pod/retail-app-deployment-68b685c7f6-8j64t	1/1	Running	0	61s
pod/retail-app-deployment-68b685c7f6-gl656	1/1	Running	0	61s
pod/retail-app-deployment-68b685c7f6-mrf6l	1/1	Running	0	61s
pod/retail-mongodb-7cd9bb8c49-fw4k5	1/1	Running	0	78s

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
service/mongodb	ClusterIP	10.100.218.198	<none>	27017/TCP	71s
service/retail-app-service	LoadBalancer	10.100.152.130	a40f909364f534bef96e01cb855ab9b7-1849244736.ap-south-1.elb.amazonaws.com	3130:30183/TCP	54s

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
deployment.apps/retail-app-deployment	4/4	4	4	61s
deployment.apps/retail-mongodb	1/1	1	1	78s

NAME	DESIRED	CURRENT	READY	AGE
replicaset.apps/retail-app-deployment-68b685c7f6	4	4	4	61s
replicaset.apps/retail-mongodb-7cd9bb8c49	1	1	1	78s

9) Open a Linux shell inside the MongoDB container and check files inside the container.

kubectl exec -it retail-mongodb-7cd9bb8c49-fw4k5 -n rithika-ns -- /bin/sh

```
# env
KUBERNETES_SERVICE_PORT=443
KUBERNETES_PORT=tcp://10.100.0.1:443
MONGODB_PORT_27017_TCP=tcp://10.100.218.198:27017
HOSTNAME=retail-mongodb-7cd9bb8c49-fw4k5
MONGO_INITDB_DATABASE=myDatabase
HOME=/data/db
GLIBC_TUNABLES=glibc.pthread.rseq=0
MONGODB_SERVICE_HOST=10.100.218.198
TERM=xterm
MONGO_PACKAGE=mongodb-org
KUBERNETES_PORT_443_TCP_ADDR=10.100.0.1
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
MONGO_MAJOR=8.2
KUBERNETES_PORT_443_TCP_PORT=443
KUBERNETES_PORT_443_TCP_PROTO=tcp
MONGODB_PORT=tcp://10.100.218.198:27017
MONGODB_SERVICE_PORT=27017
MONGODB_PORT_27017_TCP_ADDR=10.100.218.198
JSYAML_VERSION=3.13.1
KUBERNETES_SERVICE_PORT_HTTPS=443
MONGODB_PORT_27017_TCP_PORT=27017
KUBERNETES_PORT_443_TCP=tcp://10.100.0.1:443
GOSU_VERSION=1.19
MONGO_REPO=repo.mongodb.org
MONGODB_PORT_27017_TCP_PROTO=tcp
KUBERNETES_SERVICE_HOST=10.100.0.1
PWD=/
JSYAML_CHECKSUM=662e32319bdd378e91f67578e56a34954b0a2e33aca11d70ab9f4826af24b941
MONGO_VERSION=8.2.5
```

10) Open the MongoDB shell directly:

kubectl exec -it retail-mongodb-7cd9bb8c49-fw4k5 -n rithika-ns -- mongosh

To view ,use the databases and shows all documents in the products collection.

```
root@ip-172-31-17-28:/home/ubuntu/fss-Retail-App_kubernetes/k8s-manifests# kubectl exec -it retail-mongodb-7cd9bb8c49-fw4k5 -n rithika-ns -- mongosh
Current Mongosh Log ID: 699a9365e6b3f8c0b68563b0
Connecting to: mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.7.0
Using MongoDB: 8.2.5
Using Mongosh: 2.7.0

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

To help improve our products, anonymous usage data is collected and sent to MongoDB periodically (https://www.mongodb.com/legal/privacy-policy).
You can opt-out by running the disableTelemetry() command.

-----
The server generated these startup warnings when booting
2026-02-22T05:18:09.555+00:00: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
2026-02-22T05:18:09.555+00:00: For customers running the current memory allocator, we suggest changing the contents of the following sysfsFile
2026-02-22T05:18:09.555+00:00: For customers running the current memory allocator, we suggest changing the contents of the following sysfsFile
2026-02-22T05:18:09.555+00:00: We suggest setting the contents of sysfsFile to 0.
-----

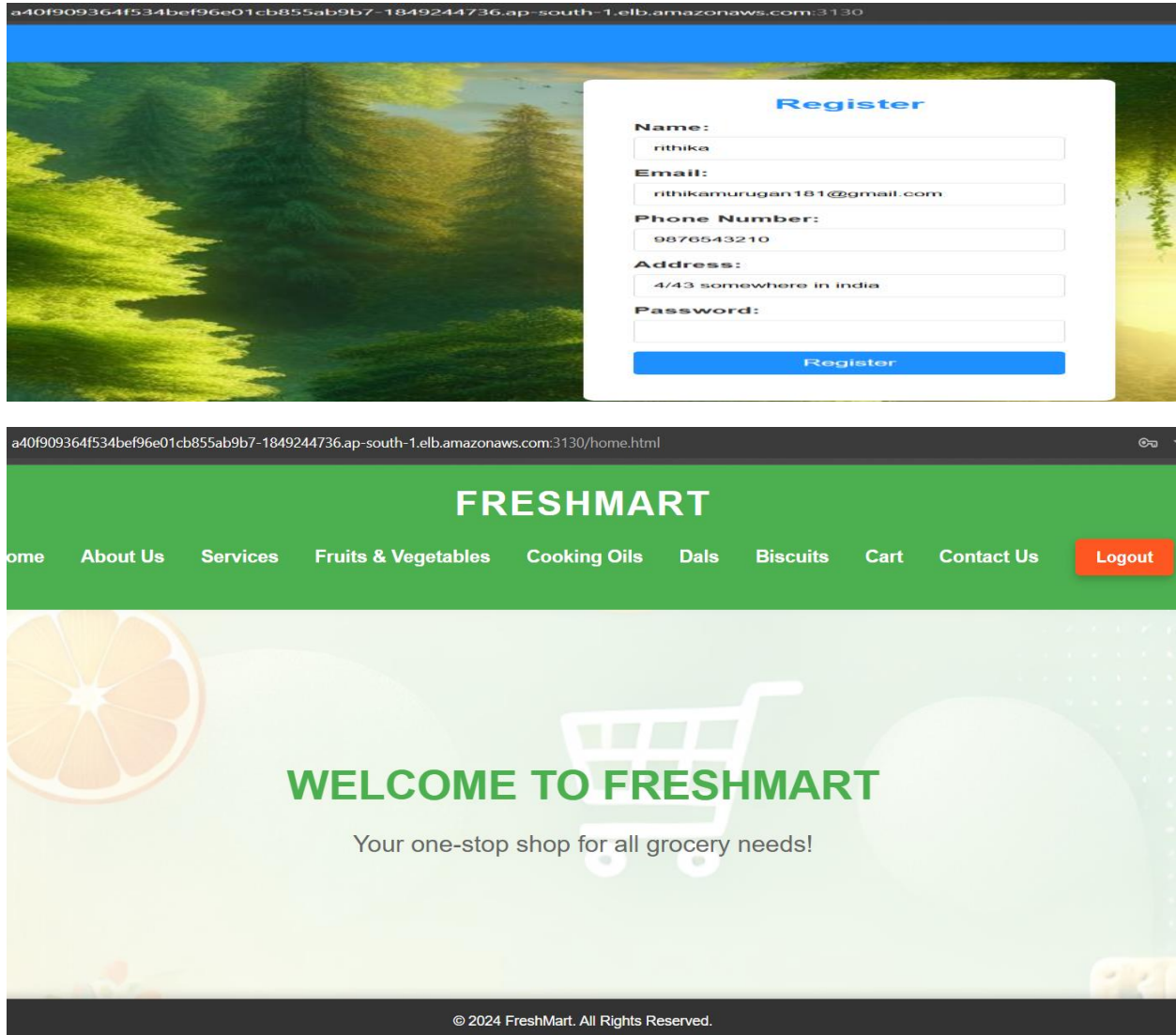
test> show dbs
admin      40.00 KiB
config     60.00 KiB
local      40.00 KiB
myDatabase 172.00 KiB
test> use myDatabase
switched to db myDatabase
myDatabase> show collections
contacts
sessions
users
myDatabase> db.users.find().pretty()
```

```
myDatabase> db.users.find().pretty()
[
  {
    _id: ObjectId('699a9267ad7917f46aa5c848'),
    name: 'rithika',
    email: 'qwerty12@gmail.com',
    phone: '9876543210',
    address: '4/43 somewhere in india',
    password: '$2a$10$J.nejJmKIpDMBZZwempVyeRBRRDbQzCY4JppogAHUNRtwzxoDZr9q',
    cart: [],
    purchaseHistory: [],
    __v: 0,
    otp: '274004',
    otpExpiry: ISODate('2026-02-22T05:27:04.195Z')
  }
]
```

11) Copy the External IP and paste it in browser:

<http://a40f909364f534bef96e01cb855ab9b7-1849244736.ap-south-1.elb.amazonaws.com:3130/>

It will get us to the home page.



SECRET YAML FILE

A Secret is a Kubernetes object used to store sensitive data such as passwords, tokens, API keys, or certificates. The data is stored in Base64-encoded format.

==> **echo -n 'chagantyteja2502@gmail.com' | base64**

- echo -n → prints the text without adding a newline
- | → sends output to the next command
- base64 → encodes the input into Base64 format

```
controlplane:~$ echo -n 'chagantyteja2502@gmail.com' | base64
Y2hhZ2FudHl0ZWphMjUwMkBnbWFpbC5jb20=
```

chagantyteja2502@gmail.com

↓

Y2hhZ2FudHl0ZWphMjUwMkBnbWFpbC5jb20=

==> **echo -n 'yxqo bjuk rdnt alzp' | base64**

```
controlplane:~$ echo -n 'yxqo bjuk rdnt alzp' | base64
eXhvcSBianVrIHJkbmQgYWx6cA==
```

yxqo bjuk rdnt alzp

↓

eXhvcSBianVrIHJkbmQgYWx6cA==

==> **echo -n 'Y2hhZ2FudHl0ZWphMjUwMkBnbWFpbC5jb20=' | base64 --decode**

==> decodes the input

```
controlplane:~$ echo -n 'Y2hhZ2FudHl0ZWphMjUwMkBnbWFpbC5jb20=' | base64 --decode
chagantyteja2502@gmail.comcontrolplane:~$
```

Y2hhZ2FudHl0ZWphMjUwMkBnbWFpbC5jb20=

↓

chagantyteja2502@gmail.com

1) Vi secret.yaml

Create secret yaml file

```
root@ip-172-31-17-28:/home/ubuntu/fss-Retail-App_kubernetes/k8s-manifests# vi secret.yaml
root@ip-172-31-17-28:/home/ubuntu/fss-Retail-App_kubernetes/k8s-manifests# ls
```

```
apiVersion: v1
kind: Secret
metadata:
  name: retail-app-secret
  namespace: rithika-ns
type: Opaque
data:
  EMAIL_USER: Y2hhZ2FudHl0ZWphMjUwMkBnbWZpbC5jb20=
  EMAIL_PASS: eXhvcSBianVrIHJkbnQgYWx6cA==
```

2) kubectl apply -f secret.yaml

Apply the file:

```
root@ip-172-31-17-28:/home/ubuntu/fss-Retail-App_kubernetes/k8s-manifests# kubectl apply -f secret.yaml
secret/retail-app-secret created
```

3) kubectl get secret -n rithika-ns

- Lists **all Secrets** in the **current namespace**
- Shows secret name, type, and age

```
root@ip-172-31-17-28:/home/ubuntu/fss-Retail-App_kubernetes/k8s-manifests# kubectl get secret -n rithika-ns
NAME                TYPE      DATA   AGE
retail-app-secret   Opaque    2       35s
```